

Week 4 Assessment Summary

Muskan

Overview

This document summarizes the completed hands-on exercises for the **Microservices** Deepskilling module. Five independent Spring Boot 3 / Spring Cloud 2023.0.1 Maven projects were built:

Module	Port	Purpose
eureka-server	8761	Service discovery registry
account-service	8081	Bank account microservice
loan-service	8082	Bank loan microservice
api-gateway	8080	Routing, logging filter, load balancing
payment-service	8083	Circuit breaker / resilience demo

Account & Loan Microservices + Eureka (hands-on 3.)

- **account-service**: GET /accounts/{number} returns { "number": ..., "type": "savings", "balance": 234343 }.
- **loan-service**: GET /loans/{number} returns { "number": ..., "type": "car", "loan": 400000, "emi": 3258, "tenure": 18 }.
- Both run on distinct ports (8081 / 8082) to avoid the port-conflict issue flagged in the hands-on when two Spring Boot apps default to 8080.
- **eureka-server**: @EnableEurekaServer on port 8761, configured with register-with-eureka=false and fetch-registry=false since it is purely a registry.
- Both business services use @EnableDiscoveryClient + spring.application.name so they register themselves and appear on the Eureka dashboard (<http://localhost:8761>).

API Gateway — Routing & Load Balancing (exercises 0. and 1.)

- **api-gateway** (Spring Cloud Gateway, WebFlux-based) routes:
 - /api/accounts/** → lb://account-service (path rewritten to /accounts/**)
 - /api/loans/** → lb://loan-service (path rewritten to /loans/**)
- The lb:// scheme delegates to Spring Cloud LoadBalancer, which distributes requests across every instance registered under that service name in Eureka — demonstrated by starting a second account-service instance on a different port and observing round-robin distribution.
- discovery.locator.enabled=true additionally exposes every registered service directly by its Eureka service ID (e.g. /account-service/accounts/1).
- **LoggingFilter**, a custom GlobalFilter (Ordered, order -1), logs the method, URI, resulting status code, and latency for every request that passes through the gateway — satisfying the “write a filter to log incoming requests” exercise.

Resilient Microservices — Circuit Breaker (exercise 1., item 4)

- **payment-service** exposes POST /payments?orderId=...&amount=...
- ThirdPartyPaymentClient simulates a slow external payment gateway (4-second latency).
- PaymentService.chargePayment(...) wraps the call with Resilience4j's @CircuitBreaker and @TimeLimiter annotations (2-second timeout, 50% failure-rate threshold, 10s open-state wait).
- fallbackCharge(...) is invoked automatically when the circuit is open or the call times out: it logs a warning with the failure reason and returns a graceful "payment pending" response instead of propagating the failure to the caller — the core resilience-pattern objective.
- /actuator/health exposes the circuit breaker's state (CLOSED / OPEN / HALF_OPEN) via management.health.circuitbreakers.enabled=true.

How to run (in order, separate terminals)

```
cd eureka-server && mvn spring-boot:run      #
http://localhost:8761
cd account-service && mvn spring-boot:run      #
http://localhost:8081
cd loan-service && mvn spring-boot:run          #
http://localhost:8082
cd api-gateway && mvn spring-boot:run          #
http://localhost:8080
cd payment-service && mvn spring-boot:run      #
http://localhost:8083
```

Sample requests

```
curl http://localhost:8081/accounts/00987987973432
curl http://localhost:8082/loans/H00987987972342

curl http://localhost:8080/api/accounts/00987987973432
curl http://localhost:8080/api/loans/H00987987972342
curl http://localhost:8080/account-service/accounts/00987987973432

curl -X POST "http://localhost:8083/payments?orderId=ORD-1&amount=500"
curl http://localhost:8083/actuator/health
```

Load-balancing demo

```
cd account-service && mvn spring-boot:run -Dspring-boot.run.arguments=--server.port=8091
```

Both instances register as account-service; repeated calls to http://localhost:8080/api/accounts/{number} are distributed across them by Spring Cloud LoadBalancer.